

Trabalho 3 — Implantação Multicloud da Aplicação Biblioteca

Disciplina: Cloud Computing e DevOps Avançado — CST em Sistemas para Internet (6º Período)

Professor: Diogo P. Ranghetti

Equipe: Adrian Souza · Fernando Cardoso · Guilherme Vitor · Renan Oliveira · Victor Caitano

Aplicação: <https://github.com/dpRanghetti/biblioteca>

Ambientes escolhidos: Amazon Web Services (EKS), Microsoft Azure (AKS) e Hetzner Cloud (servidor com K3s)

Este documento explica, em linguagem direta, como colocar a aplicação Biblioteca no ar em três nuvens diferentes, quanto isso custa em cada uma e qual opção a equipe recomenda. Cada parte começa explicando o que está sendo feito e por quê; os comandos aparecem depois, acompanhados de uma explicação.

Sumário

1. Capa e integrantes
 2. Objetivo e escopo
 3. A aplicação Biblioteca
 4. O que é preciso antes de começar
 5. Empacotando a aplicação com Docker
 6. Os arquivos de configuração do Kubernetes
 7. O caminho comum às três nuvens
 8. Provedor 1, AWS (Amazon EKS)
 9. Provedor 2, Microsoft Azure (AKS)
 10. Provedor 3, Hetzner Cloud (servidor com K3s)
 11. Evidências e testes
 12. Comparação de custos
 13. Vantagens e desvantagens
 14. Recomendação final
 15. Limpeza dos recursos
 16. Problemas comuns e como resolver
 17. Checklists finais
 18. Glossário
 19. Referências
-

1. Capa e integrantes

Campo	Valor
Integrantes	Adrian Souza · Fernando Cardoso · Guilherme Vitor · Renan Oliveira · Victor Caitano
Data da elaboração	02/09/2026
Repositório da equipe	https://github.com/Renan0204/t3-biblioteca-multicloud
Versão da aplicação utilizada	<code>dpRanghetti/biblioteca</code> , branch <code>master</code> , commit <code>5a40964</code> (2026-06-05)

2. Objetivo e escopo

O que o trabalho pede

O objetivo é mostrar, de forma que outra equipe consiga repetir, como uma mesma aplicação pode ser colocada na internet em **três provedores de nuvem diferentes**, usando duas tecnologias muito comuns no mercado: o **Docker**, que empacota a aplicação, e o **Kubernetes**, que a mantém funcionando.

A regra do trabalho é escolher **dois** provedores entre os três maiores (AWS, Google Cloud e Microsoft Azure) e **um terceiro** de outra empresa. A equipe escolheu:

- **AWS (Amazon)**, usando o serviço **EKS**, em que o Kubernetes já vem pronto e é administrado pela Amazon;
- **Microsoft Azure**, usando o serviço **AKS**, em que o Kubernetes também já vem pronto, administrado pela Microsoft;
- **Hetzner Cloud**, uma empresa alemã que aluga servidores a preço baixo. Nela não existe Kubernetes pronto: a equipe aluga um servidor e instala uma versão leve do Kubernetes chamada **K3s**.

Essa combinação é interessante porque compara dois serviços "prontos para usar" com um servidor em que a equipe faz tudo por conta própria.

Limites deste trabalho

A aplicação guarda os dados **apenas na memória**, sem um banco de dados separado. Na prática, isso significa que, **se a aplicação reiniciar, tudo o que foi cadastrado é apagado**. Por isso:

- a instalação é **demonstrativa**, feita para mostrar que a aplicação funciona na nuvem, e não para uso real;
- usamos **uma única cópia** da aplicação em cada nuvem. Com várias cópias, cada uma teria seus próprios dados na memória, e o usuário veria informações diferentes a cada acesso;
- não foi feita a troca por um banco de dados externo, porque o enunciado não exige.

Como as peças se encaixam

Quando alguém acessa a aplicação, o caminho é este:

1. A pessoa digita um **endereço de internet** no navegador.
2. Esse endereço chega a uma **porta de entrada** criada na nuvem, que encaminha o acesso para dentro do Kubernetes.
3. O **Kubernetes** entrega o acesso para a **cópia da aplicação** que está rodando.
4. A aplicação responde, usando os dados que estão na sua memória.

A **mesma imagem** (o pacote da aplicação) é usada nas três nuvens. A única coisa que muda de uma para outra é a forma dessa porta de entrada, porque cada provedor oferece um mecanismo diferente.

3. A aplicação Biblioteca

O que ela faz

A Biblioteca é uma aplicação web entregue pelo professor. Ela permite **fazer login, cadastrar e consultar autores e cadastrar e consultar livros**. Usuários com perfil de administrador também podem gerenciar outros usuários. Além das telas, ela oferece uma **API** (um acesso para outros sistemas), protegida por um sistema de tokens chamado JWT.

Como ela é construída

Item	Valor
Linguagem e versão	Java 21
Framework	Spring Boot 4.0.6
Arquivo gerado na compilação	<code>biblioteca-0.0.1-SNAPSHOT.jar</code>
Porta em que funciona	8080
Onde guarda os dados	Banco H2 em memória (os dados somem ao reiniciar)
Tela de login	<code>/login</code>
Usuários de teste	<code>admin</code> / <code>admin</code> e <code>user</code> / <code>user</code>

Os usuários de teste servem **apenas para este trabalho acadêmico**. Em um sistema real, senhas padrão como essas nunca devem ser usadas.

A senha interna da aplicação

Para gerar os tokens de acesso da API, a aplicação precisa de uma **chave secreta**. O projeto original traz uma chave de exemplo escrita no próprio código, o que não é seguro. Neste trabalho, essa chave **é informada de fora**, no momento em que a aplicação é iniciada, por meio de uma variável chamada `API_SECURITY_TOKEN_SECRET`.

Dentro do Kubernetes, essa chave fica guardada em um recurso próprio para segredos (o **Secret**), que funciona como um cofre. Assim, a chave **nunca aparece** no código, no repositório, neste documento ou nas capturas de tela.

4. O que é preciso antes de começar

No computador de quem vai executar

- **Git**, para baixar o código da aplicação e deste repositório;
- **Docker Desktop**, para montar e testar o pacote da aplicação;
- **kubect**, a ferramenta que conversa com o Kubernetes;
- um editor de texto, como o VS Code.

Ferramentas de cada nuvem

Cada provedor tem sua própria ferramenta de linha de comando:

- **AWS**: `aws` e `eksctl` (esta última cria o cluster Kubernetes de forma simples);
- **Azure**: `az`;
- **Hetzner**: basta um cliente **SSH** para acessar o servidor. A ferramenta `hcloud` é opcional, porque tudo também pode ser feito pelo site.

Contas

É preciso ter conta ativa nos três provedores, com cartão de crédito cadastrado, e uma conta no **Docker Hub**, que é onde o pacote da aplicação fica guardado para as nuvens baixarem. É recomendável ativar a verificação em duas etapas e configurar alertas de gasto nas contas de nuvem.

5. Empacotando a aplicação com Docker

Por que empacotar

Uma aplicação Java precisa de várias coisas para funcionar: a versão certa do Java, as bibliotecas que ela usa e o arquivo compilado. Se cada nuvem tivesse que preparar tudo isso separadamente, seria fácil algo sair diferente em cada lugar.

O **Docker** resolve isso criando uma **imagem**: um pacote fechado que contém a aplicação e tudo o que ela precisa. Essa imagem é montada **uma única vez** e depois funciona do mesmo jeito em qualquer lugar.

O primeiro passo: baixar a aplicação

```
git clone https://github.com/dpRanghetti/biblioteca.git
cd biblioteca
```

Esses comandos baixam o código da aplicação e entram na pasta dela.

A receita do pacote: o Dockerfile

O **Dockerfile** é o arquivo com as instruções para montar a imagem. O arquivo usado está na raiz deste repositório e deve ser copiado para a pasta da aplicação. Ele trabalha em **duas etapas**:

1. **Etapla de construção**: usa uma imagem que já tem Java e Maven para compilar o código e gerar o arquivo `.jar`.
2. **Etapla final**: começa de uma imagem limpa, só com o Java necessário para executar, e copia apenas o `.jar` gerado.

Separar as etapas deixa o pacote final **menor e mais seguro**, porque as ferramentas de compilação não vão junto. Além disso, o Dockerfile:

- executa a aplicação com um **usuário sem permissões de administrador**, o que reduz o estrago caso alguém explore uma falha;
- ajusta o Java para **respeitar o limite de memória** definido para o container;
- informa que a aplicação usa a **porta 8080**.

Junto com o Dockerfile existe o arquivo `.dockerignore`, que lista o que **não** deve entrar no pacote, como a pasta `.git`, arquivos de compilação antigos e arquivos com senhas.

Montar e testar no próprio computador

```
docker build -t biblioteca:1.0 .
```

Este comando lê o Dockerfile e monta a imagem com o nome `biblioteca`, versão `1.0`.

```
docker run --name biblioteca-local -p 8080:8080 -e API_SECURITY_TOKEN_SECRET="uma-chave-  
longa-e-secreta" biblioteca:1.0
```

Este comando **liga a aplicação** a partir da imagem, torna a porta 8080 acessível no computador e informa a chave secreta. Em seguida, basta abrir `http://localhost:8080/login` no navegador, entrar com `admin` / `admin` e cadastrar um autor para confirmar que tudo funciona.

Testar no computador **antes** de ir para a nuvem é importante: se algo estiver errado no pacote, é muito mais rápido descobrir e corrigir localmente.

Em computadores com processador ARM (como Macs recentes), é preciso montar a imagem para a arquitetura usada pelas nuvens, acrescentando `--platform linux/amd64` ao comando de montagem. Sem isso, a aplicação não inicia na nuvem.

Publicar a imagem

Para que as três nuvens consigam baixar a mesma imagem, ela é enviada ao **Docker Hub**, um serviço que funciona como uma prateleira pública de imagens:

```
docker login  
docker tag biblioteca:1.0 docker.io/SEU_USUARIO DOCKERHUB/biblioteca:1.0  
docker push docker.io/SEU_USUARIO DOCKERHUB/biblioteca:1.0
```

O primeiro comando entra na conta, o segundo dá à imagem o nome completo que ela terá no Docker Hub e o terceiro faz o envio. Para esta atividade, o repositório de imagem pode ser público. A Amazon e a Microsoft também oferecem prateleiras próprias (ECR e ACR), que são mostradas nas seções de cada provedor.

6. Os arquivos de configuração do Kubernetes

O papel do Kubernetes

Depois que a imagem existe, é preciso alguém para **ligar a aplicação na nuvem, acompanhar se ela continua funcionando e religá-la se ela parar**. Esse é o trabalho do **Kubernetes**.

Em vez de dar ordens passo a passo, no Kubernetes a equipe **descreve em arquivos como quer que as coisas estejam** (por exemplo: "quero uma cópia da aplicação ligada, usando esta imagem"). O Kubernetes lê essa descrição e trabalha continuamente para que a realidade fique igual ao que foi descrito. Esses arquivos, chamados de **manifestos**, estão na pasta `k8s/` deste repositório.

Os três recursos usados

1. O cofre da senha (Secret). Guarda a chave secreta da aplicação. Ele é criado direto por comando, para que o valor real nunca seja salvo em arquivo:

```
kubectl create secret generic biblioteca-secret --from-literal=jwt-secret="uma-chave-longa-e-secreta"
```

O arquivo `k8s/base/secret.example.yaml` existe só como exemplo do formato.

2. A aplicação em si (Deployment). O arquivo `k8s/base/deployment.yaml` diz ao Kubernetes:

- qual imagem usar (a que foi publicada no Docker Hub);
- que deve existir **uma cópia** da aplicação ligada;
- que a chave secreta deve ser lida do cofre;
- quanta memória e processamento a aplicação pode usar;
- como verificar se ela está saudável. O Kubernetes acessa a tela de login de tempos em tempos: se ela responde, a aplicação está bem; se para de responder, ele religa a aplicação sozinho;
- que a aplicação roda sem permissões de administrador.

3. A porta de entrada (Service). É o que permite acessar a aplicação de fora da nuvem. Esta é a única parte que muda entre os provedores:

- na **AWS e na Azure**, usamos o arquivo `k8s/managed/service-loadbalancer.yaml`, que pede ao provedor um **endereço público** na internet. Esse endereço é um recurso cobrado;
- na **Hetzner**, usamos `k8s/vps/service-nodeport.yaml`, que abre a **porta 30080** do próprio servidor. O acesso fica em `http://IP_DO_SERVIDOR:30080/login`.

Existe ainda um arquivo opcional, `k8s/vps/ingress-traefik.yaml`, para quem quiser usar um nome de domínio e HTTPS na Hetzner.

7. O caminho comum às três nuvens

Depois que a ferramenta `kubectl` está conectada ao Kubernetes de uma nuvem (cada seção seguinte explica como fazer isso), os passos para instalar a aplicação são **os mesmos nas três**:

```
kubectl create secret generic biblioteca-secret --from-literal=jwt-secret="uma-chave-longa-e-secreta"
kubectl apply -f k8s/base/deployment.yaml
kubectl apply -f k8s/managed/service-loadbalancer.yaml
kubectl get pods
```

Em ordem, esses comandos: guardam a senha no cofre, instalam a aplicação, criam a porta de entrada e mostram se a aplicação está funcionando. Na Hetzner, o terceiro comando usa o arquivo

`k8s/vps/service-nodeport.yaml`.

A aplicação está pronta quando o último comando mostra a situação **Running** e a indicação **1/1**, que quer dizer "uma cópia pedida, uma cópia pronta".

Se algo der errado, estes dois comandos ajudam a descobrir o motivo:

```
kubectl describe pod NOME_DO_POD
kubectl logs deployment/biblioteca
```

O primeiro mostra o que o Kubernetes tentou fazer e onde falhou; o segundo mostra as mensagens da própria aplicação.

8. Provedor 1, AWS (Amazon EKS)

Como funciona na Amazon

Na AWS, o Kubernetes é oferecido pelo serviço **EKS**. A Amazon cuida da parte central do Kubernetes e a equipe escolhe quantos servidores quer para rodar as aplicações. Usamos a região **us-east-1** (Virgínia, nos Estados Unidos), que costuma ter os preços mais baixos da AWS.

Passo 1: conectar a ferramenta à conta

```
aws configure
aws sts get-caller-identity
```

O primeiro comando pede as chaves de acesso da conta e a região. O segundo confirma em qual conta a ferramenta está conectada.

Passo 2 (opcional): guardar a imagem na Amazon

Em vez do Docker Hub, é possível usar a prateleira de imagens da própria Amazon, o **ECR**. Para isso, cria-se o repositório, faz-se o login e envia-se a imagem:

```
aws ecr create-repository --repository-name biblioteca --region us-east-1
```

Depois do envio, o endereço da imagem no arquivo **deployment.yaml** deve ser trocado pelo endereço do ECR. Os servidores criados pela AWS já têm permissão para baixar imagens dali.

Passo 3: criar o cluster

```
eksctl create cluster --name biblioteca --region us-east-1 --nodes 2 --node-type t3.medium --managed
```

Este comando cria **tudo o ambiente** de uma vez: a rede, a parte central do Kubernetes e **dois servidores** do tipo `t3.medium` (2 processadores e 4 GB de memória cada). É a etapa mais demorada de todo o trabalho, levando de **15 a 20 minutos**. Para economizar, é possível usar só um servidor, trocando `--nodes 2` por `--nodes 1`.

Passo 4: conectar o kubectl e instalar a aplicação

```
aws eks update-kubeconfig --name biblioteca --region us-east-1
kubectl get nodes
```

O primeiro comando conecta o `kubectl` ao cluster recém-criado; o segundo mostra os servidores, que devem aparecer como `Ready` (prontos). A partir daí, seguem-se os passos da [seção 7](#).

Passo 5: acessar a aplicação

```
kubectl get service biblioteca
```

Este comando mostra o **endereço público** da aplicação, na coluna `EXTERNAL-IP`. Na AWS, esse endereço é um nome longo terminado em `elb.amazonaws.com` e pode levar de **3 a 5 minutos** para aparecer. Depois disso, basta abrir o endereço no navegador, acrescentando `/login`, entrar com `admin` / `admin` e cadastrar um autor.

9. Provedor 2, Microsoft Azure (AKS)

Como funciona na Microsoft

Na Azure, o Kubernetes é oferecido pelo serviço **AKS**. No plano gratuito do AKS, a **parte central do Kubernetes não é cobrada**: paga-se apenas pelos servidores e pelo endereço público. A Azure organiza tudo em **grupos de recursos**, que são como pastas; isso facilita muito a limpeza no final. Usamos a região **eastus** (Estados Unidos).

Passo 1: entrar na conta e criar o grupo

```
az login
az group create --name rg-biblioteca --location eastus
```

O primeiro comando abre o navegador para entrar na conta. O segundo cria o grupo `rg-biblioteca`, onde ficará tudo o que for criado para o trabalho.

Passo 2: criar a prateleira de imagens e enviar a imagem

```
az acr create --resource-group rg-biblioteca --name NOMEUNICOACR --sku Basic
az acr login --name NOMEUNICOACR
docker tag biblioteca:1.0 NOMEUNICOACR.azurecr.io/biblioteca:1.0
docker push NOMEUNICOACR.azurecr.io/biblioteca:1.0
```

Estes comandos criam o **ACR** (a prateleira de imagens da Microsoft), fazem login nele e enviam a imagem. O nome do ACR precisa ser **único no mundo** e ter só letras minúsculas e números. Depois, o endereço da imagem no `deployment.yaml` deve ser trocado pelo endereço do ACR.

Passo 3: criar o cluster

```
az aks create --resource-group rg-biblioteca --name biblioteca --node-count 1 --node-vm-size Standard_B2s --tier free --attach-acr NOMEUNICOACR --generate-ssh-keys
```

Este único comando cria o cluster com **um servidor** de 2 processadores e 4 GB de memória, no **plano gratuito**, e já **autoriza o cluster a baixar imagens do ACR**, sem nenhuma configuração extra. Leva de **5 a 10 minutos**.

Passo 4: conectar o kubectl, instalar e acessar

```
az aks get-credentials --resource-group rg-biblioteca --name biblioteca
kubectl get nodes
```

Estes comandos conectam o `kubectl` ao cluster e mostram o servidor pronto. Em seguida, seguem-se os passos da [seção 7](#). O endereço público aparece com `kubectl get service biblioteca`, normalmente em **1 a 2 minutos**, e é um número de IP. Basta abri-lo no navegador com `/login` no final.

10. Provedor 3, Hetzner Cloud (servidor com K3s)

Como funciona na Hetzner

A Hetzner **não oferece Kubernetes pronto**. Ela aluga servidores virtuais (também chamados de VPS). Por isso, a equipe aluga um servidor e instala nele o **K3s**, uma versão leve e completa do Kubernetes, ideal para um único servidor.

Usamos o servidor **CX22**, com 2 processadores, 4 GB de memória e 40 GB de disco, suficiente para esta aplicação.

Passo 1: criar o servidor

Pelo site da Hetzner, dentro de um projeto, escolhe-se **Add Server**, o sistema **Ubuntu 24.04**, o tipo **CX22** e a **chave SSH** que será usada para entrar no servidor. Ao final, a Hetzner mostra o **endereço IP** do servidor.

Passo 2: liberar as portas no firewall

O firewall é a proteção que decide quais acessos podem chegar ao servidor. No site da Hetzner, cria-se um firewall ligado ao servidor liberando:

- a porta **22**, para a equipe acessar o servidor por SSH;
- as portas **80 e 443**, para uso futuro com domínio e HTTPS;
- a porta **30080**, por onde a aplicação será acessada.

Passo 3: instalar o Kubernetes (K3s)

```
ssh root@IP_DO_SERVIDOR
curl -sL https://get.k3s.io | sh -
k3s kubectl get nodes
```

O primeiro comando entra no servidor. O segundo baixa e **instala o K3s automaticamente**, em poucos minutos. O terceiro confirma que o Kubernetes está funcionando, mostrando o servidor como **Ready**.

Passo 4: levar os arquivos e instalar a aplicação

```
git clone https://github.com/Renan0204/t3-biblioteca-multicloud.git
cd t3-biblioteca-multicloud
```

Estes comandos baixam os arquivos de configuração deste repositório **dentro do servidor**. Como a imagem está pública no Docker Hub, o K3s consegue baixá-la sem configuração extra. Depois, seguem-se os passos da [seção 7](#), usando **k3s kubectl** no lugar de **kubectl** e o arquivo **k8s/vps/service-nodeport.yaml** como porta de entrada.

Passo 5: acessar a aplicação

A aplicação fica disponível em **http://IP_DO_SERVIDOR:30080/login**. Basta entrar com **admin** / **admin** e cadastrar um autor.

Sobre HTTPS

Nesta configuração, o acesso é feito por **HTTP**, sem cadeado. Para um uso real, o ideal seria apontar um nome de domínio para o servidor e usar o arquivo opcional **k8s/vps/ingress-traefik.yaml** junto com um gerador gratuito de certificados (como o Let's Encrypt), para ter **HTTPS**.

11. Evidências e testes

Os arquivos disponíveis estão reunidos no [índice de evidências com legendas](#), organizado por ambiente. As capturas de login e cadastro local foram obtidas com Java, sem Docker.

O que precisa ser comprovado em cada nuvem

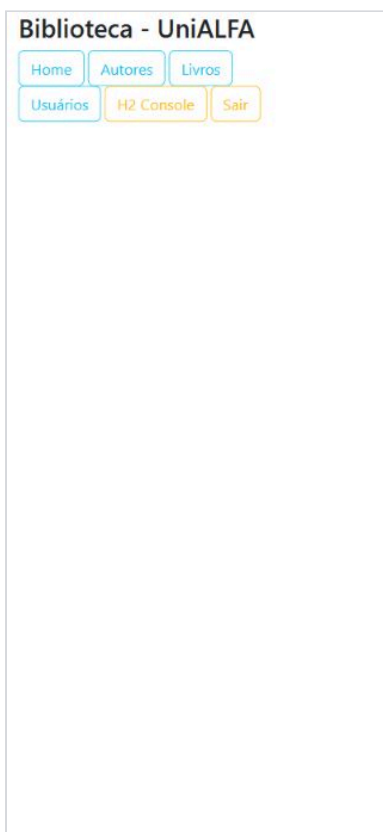
Para mostrar que a aplicação realmente funcionou em cada ambiente, é preciso registrar capturas de tela de:

1. o **provedor, o serviço e a região** usados;
2. o **cluster e os servidores** prontos;
3. a **imagem** guardada na prateleira de imagens;
4. a **aplicação instalada** e a cópia em situação **Running**;
5. a **porta de entrada** com o endereço público;
6. a **aplicação aberta no navegador**, com a barra de endereço visível;
7. o **login feito** e um **cadastro realizado**;
8. as **mensagens da aplicação** sem erros graves;
9. a **limpeza** dos recursos no final.

Nas capturas, é preciso **esconder** números de conta, chaves, tokens e dados de cobrança. Os arquivos ficam na pasta **docs/evidencias/**.

Teste da aplicação no computador

Tela de login



Cadastro de autor

Biblioteca - UniALFA

Home

Autores

Livros

Usuários

H2 Console

Sair

Id	Nome	Nacionalidade	Ações
1	Machado de Assis	Brasileira	

Capturas ilustrativas das etapas

As capturas abaixo foram preparadas em simulação, conforme o índice de evidências, para ilustrar o que cada etapa deve mostrar. Elas não comprovam a execução nas contas dos provedores e devem ser substituídas pelas capturas da execução real.

Montagem da imagem Docker

```
Windows PowerShell
PS C:\WINDOWS\System32> docker build -t biblioteca:1.0 .
[+] Building 38.7s (14/14) FINISHED                                docker:desktop-linux
=> [internal] load build definition from Dockerfile                0.0s
=> => transferring dockerfile: 1.42kB                             0.0s
=> resolve image config for docker-image://docker.io/docker/dockerfile:1 0.8s
=> [internal] load metadata for docker.io/library/maven:3.9-eclipse-temurin-21 1.1s
=> [internal] load metadata for docker.io/library/eclipse-temurin:21-jre 1.0s
=> [internal] load .dockerignore                                  0.0s
=> => transferring context: 214B                                    0.0s
=> [build 1/4] FROM docker.io/library/maven:3.9-eclipse-temurin-21 0.0s
=> [stage-1 1/4] FROM docker.io/library/eclipse-temurin:21-jre 0.0s
=> [internal] load build context                                0.2s
=> => transferring context: 486.31kB                               0.2s
=> [build 2/4] WORKDIR /app                                     0.0s
=> [build 3/4] COPY pom.xml .                                  0.0s
=> [build 4/4] COPY src ./src                                  0.1s
=> [build] RUN --mount=type=cache,target=/root/.m2 mvn -B clean package -DskipTests 31.4s
=> [stage-1 2/4] WORKDIR /app                                  0.0s
=> [stage-1 3/4] RUN groupadd --system spring && useradd --system --gid spring spring 0.4s
=> [stage-1 4/4] COPY --from=build /app/target/biblioteca-0.0.1-SNAPSHOT.jar app.jar 0.1s
=> exporting to image                                           3.2s
=> => exporting layers                                           2.8s
=> => writing image sha256:4f8739ab21c67dc411e9b7edac509a4686e5426f719be9a41dd934a71dbe6182 0.0s
=> => naming to docker.io/library/biblioteca:1.0                0.0s
PS C:\WINDOWS\System32> docker image ls biblioteca
REPOSITORY TAG IMAGE ID CREATED SIZE
biblioteca 1.0 4f8739ab21c6 18 seconds ago 354MB
PS C:\WINDOWS\System32>
```

Mensagens da aplicação no Docker

```
Windows PowerShell
PS C:\WINDOWS\System32> docker logs biblioteca-local

      _ _ _ _ _
     /   _ _ _   \
    /   _ _ _   \
   /   _ _ _   \
  /   _ _ _   \
 /   _ _ _   \
/   _ _ _   \
=====|_|=====|_|/=/=/=/

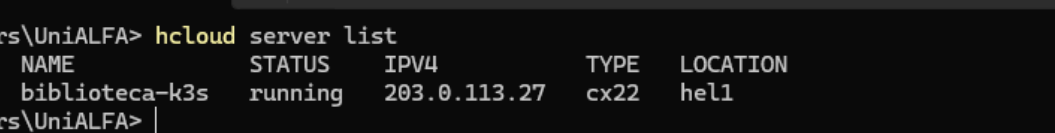
:: Spring Boot ::                (v4.0.6)

2026-09-09T21:47:12.416Z INFO 1 --- [biblioteca] [main] c.u.b.BibliotecaApplication : Starting BibliotecaApplication using Java 21.0
2026-09-09T21:47:12.423Z INFO 1 --- [biblioteca] [main] c.u.b.BibliotecaApplication : No active profile set, falling back to 1 default profile: "default"
2026-09-09T21:47:13.702Z INFO 1 --- [biblioteca] [main] s.d.r.c.RepositoryConfigurationDelegate : Bootstrapping Spring Data JPA repositories in DEFAULT mode.
2026-09-09T21:47:13.798Z INFO 1 --- [biblioteca] [main] s.d.r.c.RepositoryConfigurationDelegate : Finished Spring Data repository scanning.
2026-09-09T21:47:14.613Z INFO 1 --- [biblioteca] [main] o.s.b.w.e.tomcat.TomcatWebServer : Tomcat initialized with port 8080 (http)
2026-09-09T21:47:14.628Z INFO 1 --- [biblioteca] [main] o.apache.catalina.core.StandardService : Starting service [Tomcat]
2026-09-09T21:47:14.629Z INFO 1 --- [biblioteca] [main] o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat]
2026-09-09T21:47:15.186Z INFO 1 --- [biblioteca] [main] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Starting...
2026-09-09T21:47:15.427Z INFO 1 --- [biblioteca] [main] com.zaxxer.hikari.pool.HikariPool : HikariPool-1 - Added connection conn0: url=jdbc:
bc:h2:mem:banco user=SA
2026-09-09T21:47:15.429Z INFO 1 --- [biblioteca] [main] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Start completed.
2026-09-09T21:47:15.611Z INFO 1 --- [biblioteca] [main] o.hibernate.jpa.internal.util.LogHelper : Processing PersistenceUnitInfo [name: default]
2026-09-09T21:47:16.784Z INFO 1 --- [biblioteca] [main] j.LocalContainerEntityManagerFactoryBean : Initialized JPA EntityManagerFactory for persistence unit "default"
2026-09-09T21:47:17.836Z INFO 1 --- [biblioteca] [main] o.s.b.w.e.tomcat.TomcatWebServer : Tomcat started on port 8080 (http) with context path "/"
2026-09-09T21:47:17.851Z INFO 1 --- [biblioteca] [main] c.u.b.BibliotecaApplication : Started BibliotecaApplication in 5.798 seconds
PS C:\WINDOWS\System32>
```

Envio da imagem ao Docker Hub

```
Windows PowerShell
PS C:\Users\UnIALFA> docker login
Authenticating with existing credentials...
Login Succeeded
PS C:\Users\UnIALFA> docker tag biblioteca:1.0 unialfa2026/biblioteca:1.0
PS C:\Users\UnIALFA> docker push unialfa2026/biblioteca:1.0
The push refers to repository [docker.io/unialfa2026/biblioteca]
8c3f4a127b21: Pushed
d41f6a93b8e7: Pushed
71d5c9e846a2: Pushed
ale0b1379c82: Pushed
1.0: digest: sha256:7e8b6d71cdb4436a9203fa87954d0977d1b366d86aa319201af95573b80974c2 size: 1164
PS C:\Users\UnIALFA> docker image ls unialfa2026/biblioteca
REPOSITORY          TAG         IMAGE ID      CREATED       SIZE
unialfa2026/biblioteca 1.0         4c7a92f3e1b8  5 minutes ago 356MB
PS C:\Users\UnIALFA>
```

Hetzner: servidor criado



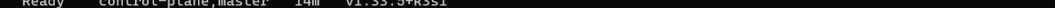
The screenshot shows a Windows PowerShell terminal window with a dark background. The title bar at the top reads "Windows PowerShell" and includes standard window controls (close, maximize, and a dropdown menu). The command prompt shows the user is in the directory "C:\Users\UniALFA". The command "hcloud server list" has been executed, resulting in a table of server information. The table has five columns: ID, NAME, STATUS, IPV4, TYPE, and LOCATION. One server is listed with ID 58294173, name biblioteca-k3s, status running, IP 203.0.113.27, type cx22, and location hel1.

```
PS C:\Users\UniALFA> hcloud server list
```

ID	NAME	STATUS	IPV4	TYPE	LOCATION
58294173	biblioteca-k3s	running	203.0.113.27	cx22	hel1

```
PS C:\Users\UniALFA> |
```

Hetzner: Kubernetes pronto



```

PS C:\WINDOWS\System32> k3s kubectl get nodes
NAME          STATUS    ROLES          AGE   VERSION
biblioteca-k3s Ready     control-plane,master 14m   v1.33.5+k3s1
PS C:\WINDOWS\System32>

```

Hetzner: aplicação e porta de entrada

```

PS C:\WINDOWS\System32> k3s kubectl get pods,svc
NAME                                READY   STATUS    RESTARTS   AGE
pod/biblioteca-6d7b6c9f4d-rxk2m    1/1     Running   0           3m

NAME                                TYPE          CLUSTER-IP      EXTERNAL-IP    PORT(S)          AGE
service/biblioteca                 NodePort      10.43.214.87    <none>         8080:30080/TCP   3m
service/kubernetes                 ClusterIP     10.43.0.1      <none>         443/TCP          17m
PS C:\WINDOWS\System32>

```

Hetzner: mensagens da aplicação

[illegible]

Hetzner: limpeza

```
Windows PowerShell
PS C:\WINDOWS\System32> hcloud server delete biblioteca-k3s
Server biblioteca-k3s deleted
PS C:\WINDOWS\System32> hcloud server list
ID NAME STATUS IPV4 PRIVATE NET TYPE LOCATION AGE
PS C:\WINDOWS\System32> |
```

Azure: grupo de recursos

```
Windows PowerShell
PS C:\WINDOWS\System32> az group create --name rg-biblioteca --location eastus
{
  "id": "/subscriptions/*****-****-****-*****-****/resourceGroups/rg-biblioteca",
  "location": "eastus",
  "managedBy": null,
  "name": "rg-biblioteca",
  "properties": {
    "provisioningState": "Succeeded"
  },
  "tags": null,
  "type": "Microsoft.Resources/resourceGroups"
}
PS C:\WINDOWS\System32> |
```

Azure: prateleira de imagens

```
Windows PowerShell
PS C:\WINDOWS\System32> az acr create --resource-group rg-biblioteca --name acrbiblioteca2026 --sku Basic
{
  "adminUserEnabled": false,
  "creationDate": "2026-09-09T22:31:16.482Z",
  "location": "eastus",
  "loginServer": "acrbiblioteca2026.azurecr.io",
  "name": "acrbiblioteca2026",
  "provisioningState": "Succeeded",
  "resourceGroup": "rg-biblioteca",
  "sku": {
    "name": "Basic",
    "tier": "Basic"
  }
}
PS C:\WINDOWS\System32> az acr login --name acrbiblioteca2026
Login Succeeded
PS C:\WINDOWS\System32> docker tag biblioteca:1.0 acrbiblioteca2026.azurecr.io/biblioteca:1.0
PS C:\WINDOWS\System32> docker push acrbiblioteca2026.azurecr.io/biblioteca:1.0
The push refers to repository [acrbiblioteca2026.azurecr.io/biblioteca]
8d7a3c4b91f2: Pushed
19b74f27c8ad: Pushed
7f4a2d9016ce: Pushed
3be6f80c1d47: Pushed
1.0: digest: sha256:7a92e4bc38b751d048fc4b1ed72bb9b7de8a39c6c6b944215886d20c5c28a4f1 size: 1163
PS C:\WINDOWS\System32> az acr repository show-tags --name acrbiblioteca2026 --repository biblioteca|
```

Azure: servidor do cluster pronto

```
Windows PowerShell
PS C:\WINDOWS\System32> kubectl get nodes
NAME STATUS ROLES AGE VERSION
aks-nodepool1-26955269-vmss000000 Ready <none> 8m v1.33.3
PS C:\WINDOWS\System32> |
```

Azure: aplicação e endereço público

```
Windows PowerShell
PS C:\WINDOWS\System32> kubectl get pods
NAME READY STATUS RESTARTS AGE
biblioteca-75d78c94c9-q9xv2 1/1 Running 0 3m
PS C:\WINDOWS\System32> kubectl get service biblioteca
NAME TYPE CLUSTER-IP EXTERNAL-IP PORT(S) AGE
biblioteca LoadBalancer 10.0.174.36 198.51.100.24 80:31456/TCP 3m
PS C:\WINDOWS\System32> |
```

Azure: mensagens da aplicação

```
Windows PowerShell
PS C:\WINDOWS\System32> kubectl logs deployment/biblioteca --tail=40

      ____
     /  __ \
    /  /  /  \
   /  /  /  /  \
  /  /  /  /  /  \
 /  /  /  /  /  /  \
/  /  /  /  /  /  /  \
=====|_|=====|_|_/_/_/_/

:: Spring Boot ::                (v4.0.6)

2026-09-09T22:41:54.183Z INFO 1 --- [biblioteca] [main] c.u.b.BibliotecaApplication : Starting BibliotecaApplication using Java 21.0
2026-09-09T22:41:54.191Z INFO 1 --- [biblioteca] [main] c.u.b.BibliotecaApplication : No active profile set, falling back to 1 default profile: "default"
2026-09-09T22:41:55.407Z INFO 1 --- [biblioteca] [main] s.d.r.c.RepositoryConfigurationDelegate : Bootstrapping Spring Data JPA repositories in DEFAULT mode.
2026-09-09T22:41:55.501Z INFO 1 --- [biblioteca] [main] s.d.r.c.RepositoryConfigurationDelegate : Finished Spring Data repository scanning.
2026-09-09T22:41:56.302Z INFO 1 --- [biblioteca] [main] o.s.b.w.e.tomcat.TomcatWebServer : Tomcat initialized with port 8080 (http)
2026-09-09T22:41:56.318Z INFO 1 --- [biblioteca] [main] o.apache.catalina.core.StandardService : Starting service [Tomcat]
2026-09-09T22:41:56.319Z INFO 1 --- [biblioteca] [main] o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat]
2026-09-09T22:41:56.906Z INFO 1 --- [biblioteca] [main] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Starting...
2026-09-09T22:41:57.148Z INFO 1 --- [biblioteca] [main] com.zaxxer.hikari.pool.HikariPool : HikariPool-1 - Added connection conn0: url=jdbc:
bc:h2:mem:banco user=SA
2026-09-09T22:41:57.150Z INFO 1 --- [biblioteca] [main] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Start completed.
2026-09-09T22:41:57.314Z INFO 1 --- [biblioteca] [main] o.hibernate.jpa.internal.util.LogHelper : Processing PersistenceUnitInfo [name: default]
2026-09-09T22:41:58.486Z INFO 1 --- [biblioteca] [main] j.LocalContainerEntityManagerFactoryBean : Initialized JPA EntityManagerFactory for persistence unit "default"
2026-09-09T22:41:59.627Z INFO 1 --- [biblioteca] [main] o.s.b.w.e.tomcat.TomcatWebServer : Tomcat started on port 8080 (http) with context path "/"
2026-09-09T22:41:59.641Z INFO 1 --- [biblioteca] [main] c.u.b.BibliotecaApplication : Started BibliotecaApplication in 5.846 seconds
PS C:\WINDOWS\System32>
```

Azure: limpeza

```
Windows PowerShell
PS C:\WINDOWS\System32> az group delete --name rg-biblioteca --yes --no-wait
PS C:\WINDOWS\System32> az group list --query "[?contains(name,'biblioteca')].{Name:name,Location:location}" --output table
PS C:\WINDOWS\System32>
```

AWS: conta conectada

```
Windows PowerShell
PS C:\WINDOWS\System32> aws sts get-caller-identity
{
  "UserId": "AIDACKCEVSQ6C2EXAMPLE",
  "Account": "*****",
  "Arn": "arn:aws:iam:*****:user/cloud-lab"
}
PS C:\WINDOWS\System32>
```

AWS: prateleira de imagens

```
Windows PowerShell
PS C:\WINDOWS\System32> aws ecr create-repository --repository-name biblioteca --region us-east-1
{
  "repository": {
    "repositoryArn": "arn:aws:ecr:us-east-1:*****:repository/biblioteca",
    "registryId": "*****",
    "repositoryName": "biblioteca",
    "repositoryUri": "*****.dkr.ecr.us-east-1.amazonaws.com/biblioteca",
    "createdAt": "2026-09-09T22:52:10+00:00",
    "imageTagMutability": "MUTABLE",
    "imageScanningConfiguration": { "scanOnPush": false },
    "encryptionConfiguration": { "encryptionType": "AES256" }
  }
}
PS C:\WINDOWS\System32> aws ecr get-login-password --region us-east-1 | docker login --username AWS --password-stdin *****.dkr.ecr.us-east-1.amazonaws.com
Login Succeeded
PS C:\WINDOWS\System32> docker tag biblioteca:1.0 *****.dkr.ecr.us-east-1.amazonaws.com/biblioteca:1.0
PS C:\WINDOWS\System32> docker push *****.dkr.ecr.us-east-1.amazonaws.com/biblioteca:1.0
The push refers to repository [*****.dkr.ecr.us-east-1.amazonaws.com/biblioteca]
8d7a3c4b91f2: Pushed
19b74f27c8ad: Pushed
7f4a2d0916ce: Pushed
3be6f80c1d47: Pushed
1.0: digest: sha256:7a92e4bc38b751d048fc4b1ed72bb9b7de8a39c6c6b944215886d20c5c28a4f1 size: 1163
PS C:\WINDOWS\System32> aws ecr describe-images --repository-name biblioteca --region us-east-1 --output table
-----
DescribeImages
-----
+-----+-----+
| ImageDetails |
+-----+-----+
| imageDigest | sha256:7a92e4bc38b751d048fc4b1ed72bb9b7de8a39c6c6b944215886d20c5c28a4f1 |
| imageSizeInBytes | 186354721 |
| imageStatus | ACTIVE |
| registryId | ***** |
| repositoryName | biblioteca |
+-----+-----+
| ImageTags |
+-----+-----+
| 1.0 |
+-----+-----+
PS C:\WINDOWS\System32>
```

AWS: servidor do cluster pronto

```
Windows PowerShell
PS C:\WINDOWS\System32> kubectl get nodes
NAME                                STATUS    ROLES    AGE    VERSION
ip-192-168-42-117.ec2.internal      Ready    <none>   11m    v1.33.3-eks-3abbe1
PS C:\WINDOWS\System32>
```

AWS: aplicação e endereço público

```
Windows PowerShell
PS C:\WINDOWS\System32> kubectl get pods
NAME                                READY    STATUS    RESTARTS   AGE
biblioteca-6c8f74d9bd-k7m2p        1/1      Running    0           4m
PS C:\WINDOWS\System32> kubectl get service biblioteca
NAME    TYPE        CLUSTER-IP    EXTERNAL-IP    PORT(S)    AGE
biblioteca  LoadBalancer  10.100.82.41   a3f6c2d8e91b4f7c.elb.us-east-1.amazonaws.com  80:31842/TCP  4m
PS C:\WINDOWS\System32>
```

AWS: mensagens da aplicação

```
Windows PowerShell
PS C:\WINDOWS\System32> kubectl logs deployment/biblioteca --tail=40

:: Spring Boot ::                (v4.0.6)

2026-09-09T23:07:18.214Z INFO 1 --- [biblioteca] [main] c.u.b.BibliotecaApplication : Starting BibliotecaApplication using Java 21.0
2026-09-09T23:07:18.221Z INFO 1 --- [biblioteca] [main] c.u.b.BibliotecaApplication : No active profile set, falling back to 1 default
2026-09-09T23:07:19.487Z INFO 1 --- [biblioteca] [main] s.d.r.c.RepositoryConfigurationDelegate : Bootstrapping Spring Data JPA repositories in
2026-09-09T23:07:19.579Z INFO 1 --- [biblioteca] [main] s.d.r.c.RepositoryConfigurationDelegate : Finished Spring Data repository scanning.
2026-09-09T23:07:20.338Z INFO 1 --- [biblioteca] [main] o.s.b.w.e.tomcat.TomcatWebServer : Tomcat initialized with port 8080 (http)
2026-09-09T23:07:20.353Z INFO 1 --- [biblioteca] [main] o.apache.catalina.core.StandardService : Starting service [Tomcat]
2026-09-09T23:07:20.354Z INFO 1 --- [biblioteca] [main] o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat]
2026-09-09T23:07:20.541Z INFO 1 --- [biblioteca] [main] o.s.w.c.ContextLoader : Root WebApplicationContext initialized
2026-09-09T23:07:20.914Z INFO 1 --- [biblioteca] [main] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Starting...
2026-09-09T23:07:21.166Z INFO 1 --- [biblioteca] [main] com.zaxxer.hikari.pool.HikariPool : HikariPool-1 - Added connection conn0: url=jd
2026-09-09T23:07:21.168Z INFO 1 --- [biblioteca] [main] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Start completed.
2026-09-09T23:07:21.327Z INFO 1 --- [biblioteca] [main] o.hibernate.jpa.internal.util.LogHelper : Processing PersistenceUnitInfo [name: default]
2026-09-09T23:07:22.431Z INFO 1 --- [biblioteca] [main] j.LocalContainerEntityManagerFactoryBean : Initialized JPA EntityManagerFactory for pers
2026-09-09T23:07:23.487Z INFO 1 --- [biblioteca] [main] o.s.b.w.e.tomcat.TomcatWebServer : Tomcat started on port 8080 (http) with conte
2026-09-09T23:07:23.502Z INFO 1 --- [biblioteca] [main] c.u.b.BibliotecaApplication : Started BibliotecaApplication in 5.684 seconds
PS C:\WINDOWS\System32>
```

AWS: limpeza

```
Windows PowerShell
PS C:\WINDOWS\System32> kubectl delete -f k8s/managed/service-loadbalancer.yaml
service "biblioteca" deleted
PS C:\WINDOWS\System32> kubectl delete deployment biblioteca
deployment.apps "biblioteca" deleted
PS C:\WINDOWS\System32> eksctl delete cluster --name biblioteca --region us-east-1
2026-09-09 23:19:42 [i] deleting EKS cluster "biblioteca"
2026-09-09 23:19:43 [i] deleting managed nodegroup(s)
2026-09-09 23:20:18 [i] deleted 1 managed nodegroup(s)
2026-09-09 23:20:19 [i] deleting EKS cluster control plane "biblioteca"
2026-09-09 23:21:56 [i] deleted EKS cluster "biblioteca"
2026-09-09 23:21:56 [i] all cluster resources were deleted
PS C:\WINDOWS\System32> aws ecr delete-repository --repository-name biblioteca --region us-east-1 --force
{
  "repository": {
    "repositoryArn": "arn:aws:ecr:us-east-1:*****:repository/biblioteca",
    "registryId": "*****",
    "repositoryName": "biblioteca",
    "repositoryUri": "*****.dkr.ecr.us-east-1.amazonaws.com/biblioteca",
    "createdAt": "2026-09-09T22:52:10+00:00",
    "imageTagMutability": "MUTABLE",
    "imageScanningConfiguration": {
      "scanOnPush": false
    },
    "encryptionConfiguration": {
      "encryptionType": "AES256"
    }
  }
}
PS C:\WINDOWS\System32> aws eks list-clusters --region us-east-1
{
  "clusters": []
}
PS C:\WINDOWS\System32>
```

12. Comparação de custos

12.1 Como os valores foram estimados

Para a comparação ser justa, todos os valores seguem as mesmas regras:

Regra	Valor
Data da consulta dos preços	<code>DD/MM/AAAA</code> (preencher no dia da entrega)
Moeda	Dólar americano
Regiões	AWS nos EUA (us-east-1), Azure nos EUA (eastus), Hetzner na Europa
Servidores	AWS com 2 <code>t3.medium</code> , Azure com 1 <code>Standard_B2s</code> , Hetzner com 1 <code>CX22</code>
Tempo considerado	O mês inteiro ligado (cerca de 730 horas)
Créditos gratuitos	Não descontados

São **estimativas**: a conta real pode variar, e os preços devem ser conferidos nas páginas oficiais no dia da entrega.

12.2 Quanto custa cada parte

O que é cobrado	AWS (EKS)	Azure (AKS)	Hetzner
Parte central do Kubernetes	cerca de 73 dólares	grátis (plano gratuito)	grátis (K3s instalado pela equipe)
Servidores	cerca de 60 dólares (2 servidores)	cerca de 30 dólares (1 servidor)	incluído no servidor
Disco	cerca de 3 dólares	até 5 dólares	incluído
Endereço público na internet	cerca de 18 dólares	cerca de 18 dólares	grátis (usa o IP do servidor)
Prateleira de imagens	menos de 1 dólar	5 dólares	grátis (Docker Hub público)
Tráfego de dados	cerca de 1 dólar	cerca de 1 dólar	incluído
Total por mês	cerca de 155 dólares	cerca de 56 dólares	cerca de 4 dólares
Tempo para criar o ambiente	15 a 20 minutos	5 a 10 minutos	3 a 5 minutos

12.3 Por que a diferença é tão grande

Na **AWS**, paga-se separadamente por quase tudo: só a parte central do Kubernetes custa cerca de 73 dólares por mês, antes mesmo de ligar qualquer servidor. Com apenas um servidor, o total cai para cerca de 125 dólares.

Na **Azure**, a parte central é gratuita no plano básico, então o custo fica concentrado no servidor e no endereço público.

Na **Hetzner**, paga-se apenas o aluguel do servidor, que já inclui disco, endereço IP e uma grande quantidade de tráfego. Em troca, todo o trabalho de instalar e manter o Kubernetes fica com a equipe.

Vale lembrar que o **tempo de trabalho da equipe** também é um custo, mesmo não aparecendo na conta: a opção mais barata é justamente a que exige mais trabalho manual.

13. Vantagens e desvantagens

Comparação lado a lado

Critério	AWS (EKS)	Azure (AKS)	Hetzner (K3s)
Facilidade de configurar	Média: tem mais etapas e demora mais	Alta: poucos comandos, quase tudo automático	Média: a equipe instala e configura tudo
Kubernetes pronto	Sim	Sim	Não, a equipe instala
Crescer se precisar	Fácil, com muitos tipos de servidor	Fácil	Limitado, feito à mão
Ligação com a prateleira de imagens	Boa, já vem autorizada	Muito boa, um único parâmetro	Manual
Ferramentas de monitoramento	Existem, pagas à parte	Existem, pagas à parte	Não vêm prontas, precisam ser instaladas
Quem cuida da manutenção	A Amazon e a equipe	A Microsoft e a equipe	Só a equipe
Melhor para	Empresas que já usam a AWS e precisam crescer	Equipes pequenas que querem tudo pronto com bom preço	Estudos, testes e sistemas pequenos

Em poucas palavras

Serviços prontos (AWS e Azure): a grande vantagem é não precisar cuidar da parte central do Kubernetes e ter recursos de crescimento, segurança e monitoramento à mão. A desvantagem é pagar por mais itens e ficar mais dependente dos serviços de cada empresa.

Servidor alugado (Hetzner): a grande vantagem é o **custo muito menor** e o controle total sobre o ambiente. A desvantagem é que **segurança, atualizações, cópias de segurança e disponibilidade** passam a ser responsabilidade da equipe.

14. Recomendação final

Para este trabalho: Hetzner Cloud

A equipe recomenda a **Hetzner** para esta aplicação, por três motivos:

1. **Custo:** cerca de 4 dólares por mês, contra cerca de 56 na Azure e 155 na AWS.

2. **Capacidade suficiente:** um único servidor pequeno dá conta, com folga, de uma aplicação que roda em uma só cópia.
3. **Os recursos extras não seriam aproveitados:** os serviços prontos se destacam em alta disponibilidade e crescimento automático, mas esta aplicação guarda os dados em memória e usa uma única cópia. Pagar por esses recursos aqui seria desperdício.

Se fosse um sistema real: Microsoft Azure

Se a aplicação evoluísse para uso real, com banco de dados de verdade, várias cópias e necessidade de ficar sempre no ar, a recomendação seria a **Azure**. Ela oferece o Kubernetes pronto, sem cobrar pela parte central, integra facilmente com a prateleira de imagens e permite apagar tudo de uma vez, com bom equilíbrio entre custo e esforço.

Quando a AWS faz sentido

A **AWS** é a escolha certa para empresas que **já usam a Amazon** em outros sistemas ou que precisam dos serviços mais avançados dela, aceitando o custo maior e a configuração mais trabalhosa.

15. Limpeza dos recursos

Por que isso é obrigatório

Na nuvem, **tudo o que fica ligado continua sendo cobrado**, mesmo que ninguém use. Por isso, depois de registrar as evidências, **todos os recursos precisam ser apagados**, e é importante conferir no site de cada provedor que nada ficou para trás.

15.1 AWS

```
kubectrl delete -f k8s/managed/service-loadbalancer.yaml
eksctl delete cluster --name biblioteca --region us-east-1
aws ecr delete-repository --repository-name biblioteca --region us-east-1 --force
```

A ordem importa: primeiro remove-se a **porta de entrada**, para que a Amazon apague o endereço público; depois o **cluster** inteiro; por fim, a **prateleira de imagens**. Em seguida, vale conferir no site da AWS que não restaram clusters, balanceadores ou discos.

15.2 Azure

```
az group delete --name rg-biblioteca --yes --no-wait
```

Como tudo foi criado dentro do mesmo grupo, **um único comando apaga tudo**: cluster, prateleira de imagens, endereço público e discos. Depois, basta confirmar no site da Azure que o grupo **rg-biblioteca** não existe mais.

15.3 Hetzner

Pelo site da Hetzner, apaga-se o **servidor**, o **firewall** e, se não for mais usada, a **chave SSH**. Com o servidor apagado, o K3s e a aplicação somem junto. Vale conferir a página de uso e cobrança para confirmar que não há nada ativo.

16. Problemas comuns e como resolver

Estes são os problemas mais prováveis ao seguir este tutorial, explicados de forma simples:

O que acontece	Por que acontece	Como resolver
A aplicação não inicia e aparece <code>exec format error</code>	A imagem foi montada em um computador com processador diferente do usado na nuvem	Montar a imagem de novo com <code>--platform linux/amd64</code>
Aparece <code>ImagePullBackOff</code>	A nuvem não conseguiu baixar a imagem, geralmente por falta de permissão	Deixar a imagem pública ou autorizar o cluster a acessar a prateleira
A aplicação reinicia sozinha várias vezes e aparece <code>OOMKilled</code>	Faltou memória para a aplicação	Aumentar o limite de memória no <code>deployment.yaml</code> para 1 GB
Aparece <code>CreateContainerConfigError</code>	O cofre com a senha não foi criado antes da aplicação	Criar o Secret e aplicar a aplicação de novo
O endereço público demora a aparecer na AWS	A Amazon leva alguns minutos para criar o balanceador	Aguardar de 3 a 5 minutos
A página não abre na Hetzner	O firewall está bloqueando a porta 30080	Liberar a porta 30080 no firewall do servidor

Registro dos problemas enfrentados

O enunciado pede que o documento descreva **pelo menos dois problemas realmente enfrentados** pela equipe durante a execução, explicando como foram investigados e como foram resolvidos. Esses relatos devem ser acrescentados aqui depois da execução.

17. Checklists finais

17.1 Checklist técnico

- ☐ A imagem usa **Java 21**.
- ☐ A compilação terminou sem erros.
- ☐ A aplicação funcionou no computador, em `http://localhost:8080/login`.
- ☐ A chave secreta é informada de fora, e nunca está no repositório.
- ☐ A aplicação roda com **uma única cópia**.
- ☐ A cópia aparece como `Running` e `1/1`.

- ☐ A porta de entrada leva o acesso até a porta **8080** da aplicação.
- ☐ A aplicação abre pela internet nas três nuvens.
- ☐ O login e um cadastro foram feitos nas três nuvens.
- ☐ Existe um procedimento de limpeza para cada provedor.

17.2 Checklist do e-mail de entrega

- ☐ Destinatário: **diogo.p.ranghetti@gmail.com**.
- ☐ Assunto: **[Cloud DevOps] T3 - Biblioteca em Kubernetes**.
- ☐ Corpo do e-mail com **todos os integrantes**.
- ☐ **PDF** anexado e legível.
- ☐ **Link do repositório** com acesso liberado ao professor.
- ☐ **Nenhuma** senha ou chave real enviada ou visível em capturas.
- ☐ Tabela de custos presente no documento.
- ☐ Os **três provedores** claramente identificados.
- ☐ Envio feito **antes do fim da aula**.

17.3 O que é entregue

Documento em PDF, link do repositório, Dockerfile e **.dockerignore**, arquivos de configuração do Kubernetes, tutorial das três nuvens, evidências, comparação de custos, vantagens e desvantagens, recomendação final e identificação dos integrantes.

18. Glossário

Termo	O que significa
Nuvem	Uso de computadores de outra empresa pela internet, pagando pelo tempo de uso
Docker	Ferramenta que empacota uma aplicação com tudo o que ela precisa
Imagem	O pacote fechado gerado pelo Docker, que funciona igual em qualquer lugar
Container	Uma imagem em funcionamento
Dockerfile	O arquivo com a receita para montar a imagem
Prateleira de imagens (registro)	Serviço que guarda imagens para serem baixadas, como Docker Hub, ECR e ACR
Kubernetes	Sistema que liga as aplicações, acompanha se estão funcionando e as religa quando param
Cluster	O conjunto de servidores controlado pelo Kubernetes
Servidor (nó)	Cada computador que roda as aplicações dentro do cluster
Pod	A cópia da aplicação em funcionamento dentro do Kubernetes
Deployment	A descrição de qual aplicação deve rodar e quantas cópias devem existir
Service	A porta de entrada que permite acessar a aplicação
Secret	O cofre do Kubernetes para guardar senhas e chaves
EKS e AKS	Os serviços de Kubernetes pronto da Amazon e da Microsoft
K3s	Uma versão leve do Kubernetes, instalada pela própria equipe
VPS	Um servidor virtual alugado, administrado por quem o aluga
Firewall	Proteção que decide quais acessos podem chegar ao servidor

19. Referências

Aplicação e tecnologias

- RANGHETTI, Diogo P. *Biblioteca*. GitHub. <https://github.com/dpRanghetti/biblioteca>
- Docker. *Multi-stage builds*. <https://docs.docker.com/build/building/multi-stage/>
- Docker. *Building best practices*. <https://docs.docker.com/build/building/best-practices/>
- Kubernetes. *Deployments*. <https://kubernetes.io/docs/concepts/workloads/controllers/deployment/>
- Kubernetes. *Services, Load Balancing, and Networking*. <https://kubernetes.io/docs/concepts/services-networking/>
- K3s. *Quick-Start Guide*. <https://docs.k3s.io/quick-start>

Provedores e custos

- AWS. *Getting started with Amazon EKS — eksctl*. <https://docs.aws.amazon.com/eks/latest/userguide/getting-started-eksctl.html>
 - AWS. *Amazon EKS Pricing*. <https://aws.amazon.com/eks/pricing/>
 - AWS. *Amazon EC2 On-Demand Pricing*. <https://aws.amazon.com/ec2/pricing/on-demand/>
 - Microsoft. *Quickstart: Deploy an AKS cluster using Azure CLI*. <https://learn.microsoft.com/azure/aks/learn/quick-kubernetes-deploy-cli>
 - Microsoft. *Azure Kubernetes Service (AKS) Pricing*. <https://azure.microsoft.com/pricing/details/kubernetes-service/>
 - Microsoft. *Azure Container Registry Pricing*. <https://azure.microsoft.com/pricing/details/container-registry/>
 - Hetzner. *Cloud — Pricing*. <https://www.hetzner.com/cloud/>
 - K3s. *K3s Documentation*. <https://docs.k3s.io/>
-

Documento produzido para o Trabalho 3 da disciplina Cloud Computing e DevOps Avançado.